

Capgemini Data Engineer Interview

2–5 Years Experience

Summary

Category	Focus	Count
ADF / ETL Design	Incremental, Validation	3
PySpark	Optimization, Join, Partition	4
SQL	Window, Aggregation	2
Delta / Storage	Schema, Governance	3
Debugging / Behavioral	Production Fix	3

1 What they asked me

How would you design a **daily incremental data pipeline** from SQL Server to Azure Data Lake?

What I said

I explained my approach in 5 steps:

1. **Extract:** Use Azure Data Factory (ADF) to connect to SQL source via Linked Service.
2. **Watermark Logic:** Use a **Lookup Activity** to fetch `LastModifiedDate` from a control table.
3. **Transform:** Filter records in Copy Activity using dynamic query:
4. `SELECT * FROM Sales WHERE ModifiedDate > @{variables('LastLoadDate')}`
5. **Load:** Write data to **ADLS** in Parquet format (for compression).
6. **Update:** Update control table after success.

Tips

Capgemini checks **data flow thinking**, not button clicks.
Use keywords: *parameterization, watermark, incremental load*.

2 What they asked me

What is the difference between **ETL** and **ELT**, and which is better in cloud-based systems?

What I said

- **ETL**: Extract → Transform → Load (Transform before loading).
 - **ELT**: Extract → Load → Transform (Transform after loading).
- In modern data engineering, **ELT** is preferred since transformations can happen inside cloud engines like **Snowflake**, **Databricks**, or **Synapse**.

Tips

Say: “Cloud enables scalable compute → ELT is faster and cost-efficient.”
Capgemini expects that insight.

3 What they asked me

How do you handle **schema drift** in Delta Lake or Data Lake ingestion?

What I said

I explained:

```
df.write.option("mergeSchema", "true") \
    .format("delta") \
    .mode("append") \
    .save(path)
```

Delta supports **schema evolution**, automatically adjusting to new columns.

Tips

Mention using **data validation before write** to avoid silent errors.
They'll often follow up: “What if a column data type changes?”

4 What they asked me

Write a SQL query to get **top 2 employees by salary in each department**.

What I said

```
SELECT name, dept, salary
FROM (
    SELECT name, dept, salary,
           DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS rnk
```

7 What they asked me

What are the **advantages of Parquet format** over CSV?

What I said

Feature	Parquet	CSV
Storage	Columnar, compressed	Row-based, large size
Query	Faster due to predicate pushdown	Slow
Schema	Embedded	Not stored

I said we prefer Parquet for **read-heavy analytical workloads**.

Tips

Mention *predicate pushdown* — it's an advanced keyword that impresses.

8 What they asked me

What is the use of **Data Flow in ADF**, and when do you prefer it over Databricks?

What I said

ADF **Data Flow** is a **visual ETL design** tool within ADF:

- For **simple transformations** (join, filter, aggregate).
- For heavy computation → use **Databricks notebooks**.
ADF Data Flow runs on an **Azure Integration Runtime cluster**.

Tips

Capgemini checks tool comparison knowledge.

Say: *ADF = low-code ETL, Databricks = code-heavy transformations*.

9 What they asked me

Explain the difference between **Managed** and **External** tables in Hive or Databricks.

```
FROM Employee
) x
WHERE rnk <= 2;
```

Tips

Capgemini loves **window function** questions.
Always mention **OVER + PARTITION BY + ORDER BY**.

5 What they asked me

Explain how you would **optimize a slow PySpark job**.

What I said

I mentioned:

- Identify **data skew** using group counts.
- Use **broadcast joins** for small lookup tables.
- **Repartition** large DataFrames:
`df = df.repartition(10, "country")`
- Cache reused DataFrames.
- Use **Parquet/Delta** for I/O efficiency.

Tips

They test if you know **Spark internals** — mention *shuffle*, *partitions*, and *Spark UI*.

6 What they asked me

How do you perform **data validation** during pipeline execution?

What I said

I said I add **data validation layer** before load:

- Validate schema and column count.
- Check nulls or invalid values:
`invalid_df = df.filter("id IS NULL OR amount <= 0")`
- Log invalid records in a **reject folder** for review.
- Maintain **audit table** with counts.

Tips

Show awareness of **data quality frameworks** (Great Expectations / rule-based checks).

Use that keyword “idempotent” confidently — it signals maturity.

12 What they asked me

Explain **Partitioning** in Spark and how it improves performance.

What I said

Partitioning splits data across executors:

```
df.write.partitionBy("region").parquet("/data/output/")
```

When queries filter by `region`, Spark reads only that partition → faster I/O.

Tips

Capgemini loves real examples: say “*partitioned by date for daily loads.*”

13 What they asked me

How do you monitor or debug a **Spark job in production**?

What I said

- Use **Spark UI** to check stage duration and skew.
- Inspect **executor memory & shuffle read/write**.
- Add **log4j** for custom logging.
- Enable **metrics via Prometheus/Grafana** for monitoring.

Tips

They expect mention of **Spark UI** → always include that.

14 What they asked me

What is the difference between **inner join** and **left anti join** in PySpark?

What I said

- **Inner Join:** returns matching records only.
- **Left Anti Join:** returns records from left that **don’t exist** in right.

```
df_left.join(df_right, "id", "left_anti")
```

What I said

- **Managed Table:** Both data & metadata controlled by system; deleting table deletes data.
- **External Table:** Only metadata is managed; data remains.

```
CREATE EXTERNAL TABLE sales (...) LOCATION '/mnt/raw/sales/';
```

Tips

They often ask: “Which one do you use for production data?” → **External table** (safe deletion).

10 What they asked me

How would you handle a **failed pipeline run** in production?

What I said

- Check **ADF Monitor** → Activity run logs.
- Identify error message (source connectivity, null reference, data mismatch).
- **Re-run** from failed activity using pipeline parameters.
- Log failure in **control table**.
- Implement **retry + alerting mechanism** for future.

Tips

Mention your **root-cause** → **fix** → **prevention** mindset.
Capgemini values structured debugging.

11 What they asked me

How do you ensure **idempotency** in data pipelines?

What I said

Idempotency = safe re-runs.
I achieve it using:

- **Merge logic** instead of insert-only writes.
- Maintain **checkpoint tables** for last processed records.
- Use **transactional writes** (Delta ACID).

Tips

Tips

Capgemini frequently asks this in incremental load context.
Say: “We use left_anti to identify new records to insert.”

15 What they asked me

Tell me about a time when your pipeline failed — how did you fix and prevent it?

What I said

A PySpark job failed due to corrupted JSON input.

- I implemented **try-except** block for parsing.
- Moved failed files to **quarantine folder**.
- Added **data validation rule** before read step.
After that, failure rate dropped by 95%.

Tips

End behavioral answers with **impact metrics** — recruiters love measurable success.